Ex.No: 06 **DATA FITTING BY REGRESSION** **Date:**

Aim:

To implement data fitting using regression analysis in MATLAB and to visualize the best-fit line for a given dataset.

Theory:

Regression analysis is a statistical technique used to model and analyze the relationship between dependent (Y) and independent (X) variables.

Linear Regression (First-Order Polynomial):

$$Y = mX + c$$

$m = Slope$

$c = Intercept$

The coefficients m, c are obtained by minimizing the **sum of squared errors** between actual and predicted values.

Polynomial Regression (Higher-Order):

$$Y = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n$$

In MATLAB, regression fitting can be done using:

$polyfit(X, Y, n) \rightarrow fits\ a\ polynomial\ of\ degree\ n$.

$polyval(p, X) \rightarrow evaluates\ fitted\ values\ at\ points\ X$.

Algorithm:

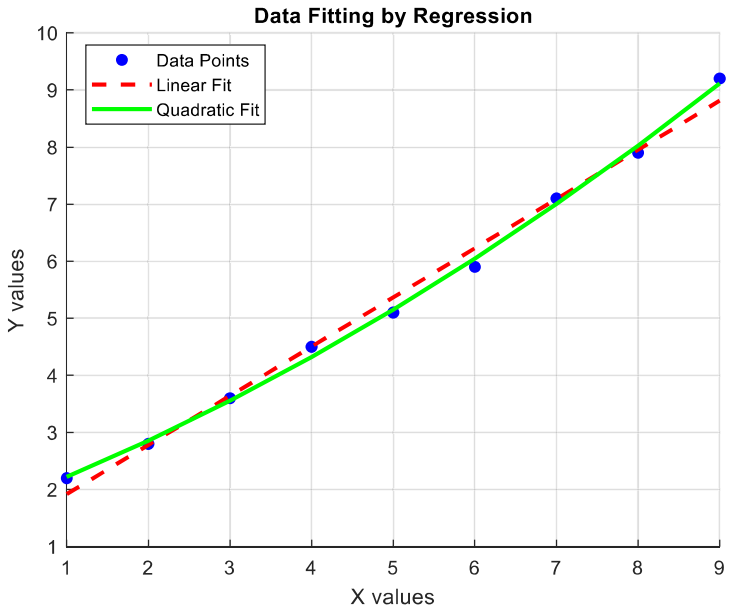
1. Start.
2. Input the dataset X, Y.
3. Choose the degree of regression (linear, quadratic, etc.).
4. Use MATLAB polyfit to compute regression coefficients.
5. Use polyval to estimate fitted Y values.
6. Plot the original data points and the fitted regression curve.
7. Stop.

Program:

```
clc;
clear;
% Step 1: Define dataset
X = [1 2 3 4 5 6 7 8 9];
```

```
Y = [2.2 2.8 3.6 4.5 5.1 5.9 7.1 7.9 9.2];
% Step 2: Linear regression (1st order polynomial)
p1 = polyfit(X, Y, 1);      % slope & intercept
Y_fit1 = polyval(p1, X);
% Step 3: Quadratic regression (2nd order polynomial)
p2 = polyfit(X, Y, 2);
Y_fit2 = polyval(p2, X);
% Step 4: Plot data and regression lines
figure;
scatter(X, Y, 'b', 'filled'); hold on;
plot(X, Y_fit1, 'r--', 'LineWidth', 2);
plot(X, Y_fit2, 'g-', 'LineWidth', 2);
title('Data Fitting by Regression');
xlabel('X values'); ylabel('Y values');
legend('Data Points','Linear Fit','Quadratic Fit','Location','best');
grid on;
```

Output:



Result:

Data fitting using regression was successfully implemented in MATLAB. Both linear and quadratic regression models were fitted to the dataset. The regression plots demonstrated that the quadratic model provided a closer fit to the data compared to the linear model.

Ex.No: 07 **IMPLEMENTATION OF COVARIANCE AND CORRELATION** **Date:**

Aim:

To implement covariance and correlation between two datasets using MATLAB and to analyze their linear relationship.

Theory:

Covariance

- **Covariance** is a statistical measure of the degree to which two random variables change together.
- If $X = \{x_1, x_2, \ldots, x_n\}$ and $Y = \{y_1, y_2, \ldots, y_n\}$, the covariance is defined as:

$$\mathrm{Cov}(X,Y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{X})(y_i - \bar{Y})$$

Where,

- $\bar{X} = mean\ of\ X$
- $\bar{Y} = mean\ of\ Y$
- $n = Number\ of\ Observations$

Interpretation:

$\boldsymbol{Cov(X,Y) > 0}$ → Positive relationship (both variables increase together).

$\boldsymbol{Cov(X,Y) < 0}$ → Negative relationship (one increases, the other decreases).

$\boldsymbol{Cov(X,Y) = 0}$ → No linear relationship.

Correlation

- The Pearson correlation coefficient is the normalized form of covariance:

$$r_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Where,

- $\sigma_X, \sigma_Y\ are\ the\ standard\ deviations\ of\ X\ and\ Y$
- $-1 \leq r_{XY} \geq 1$

Interpretation:

$\boldsymbol{r = +1}$ → Perfect positive correlation

$\boldsymbol{r = -1}$ → Perfect negative correlation

$\boldsymbol{r = 0}$ → No linear correlation

Algorithm:

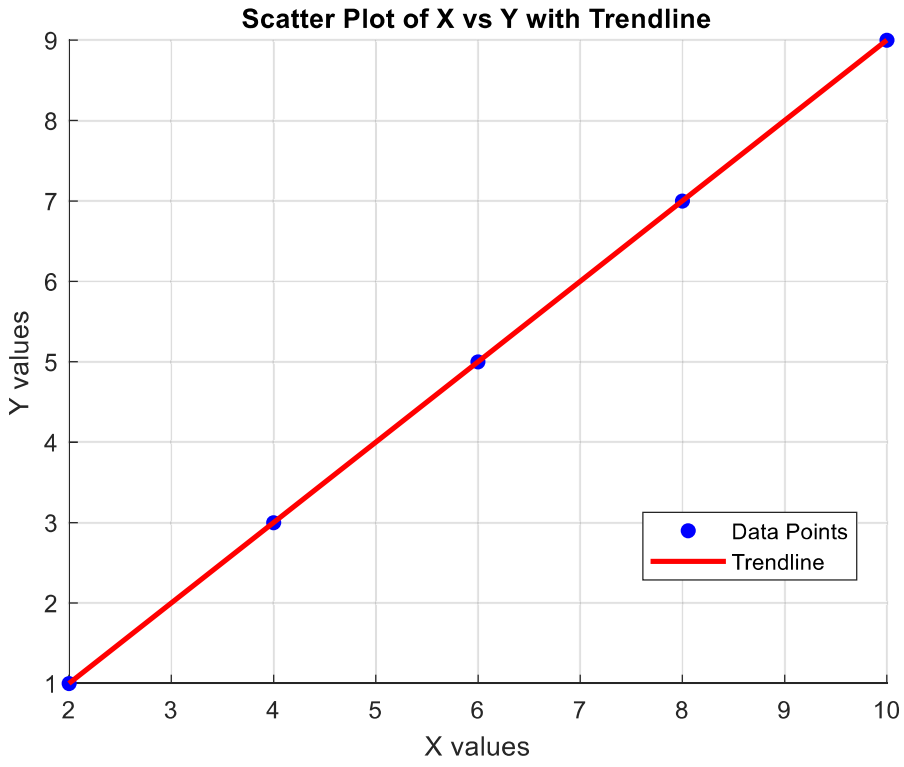
1. Start.
2. Input two datasets X and Y.
3. Compute the means of X and Y
4. Compute covariance using the formula.
5. Compute correlation using covariance and standard deviations.
6. Display results and plot scatter with trendline.
7. Stop.

Program:

```
clc;
clear;
% Step 1: Define datasets
X = [2 4 6 8 10];
Y = [1 3 5 7 9];
% Step 2: Covariance (manual)
n = length(X);
mean_X = mean(X);
mean_Y = mean(Y);
cov_XY = sum((X - mean_X) .* (Y - mean_Y)) / (n - 1);
% Step 3: Standard deviations
std_X = std(X);
std_Y = std(Y);
% Step 4: Correlation (manual)
corr_XY = cov_XY / (std_X * std_Y);
% Step 5: Display results
disp(['Covariance = ', num2str(cov_XY)]);
disp(['Correlation = ', num2str(corr_XY)]);
% Step 6: Cross-check using MATLAB functions
cov_matrix = cov(X, Y);
corr_matrix = corrcoef(X, Y);
disp('Covariance matrix:');
disp(cov_matrix);
```

```
disp('Correlation matrix:');
disp(corr_matrix);
% Step 7: Scatter plot with trendline
figure;
scatter(X, Y, 'filled','b');
hold on;
p = polyfit(X, Y, 1);    % linear fit
y_fit = polyval(p, X);
plot(X, y_fit, 'r', 'LineWidth', 2);
title('Scatter Plot of X vs Y with Trendline');
xlabel('X values');
ylabel('Y values');
legend('Data Points','Trendline','Location','best');
grid on;
```

Output:



Covariance = 10

Correlation = 1

Covariance matrix:

10 10

10 10

Correlation matrix:

1.0000 1.0000

1.0000 1.0000

Results:

The covariance and correlation between the given datasets were successfully computed in MATLAB. The covariance value of **10** indicates a positive relationship. The correlation coefficient of **1** shows a perfect positive linear relationship between X and Y. The scatter plot with a trendline confirmed this linear dependency.

Ex.No: 08 **IMPLEMENTATION OF CRISP MODEL** **Date:**

Aim:

To implement and study the operations of **crisp sets** (union, intersection, difference, and complement) using MATLAB.

Theory:

A **crisp set** is a collection of well-defined distinct objects, where an element either **belongs** to the set or **does not belong**.

The membership function of a crisp set is binary:

$$\mu_A(x) = \begin{cases} 1, & x \in A \\ 0, & x \notin A \end{cases}$$

Crisp Set Operations:

Union:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

Intersection:

$$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$$

Difference:

$$A - B = \{x \mid x \in A \text{ and } x \notin B\}$$

Complement:

$$\bar{A} = \{x \mid x \notin A\}$$

Algorithm:

1. Start.
2. Define the universe of discourse U.
3. Define crisp sets A and B as subsets of U.
4. Perform:
   - Union → combine elements of both sets.
   - Intersection → common elements of both sets.
   - Difference → elements of one set not in the other.
   - Complement → elements in U but not in the set.

5. Display the results.
6. Stop.

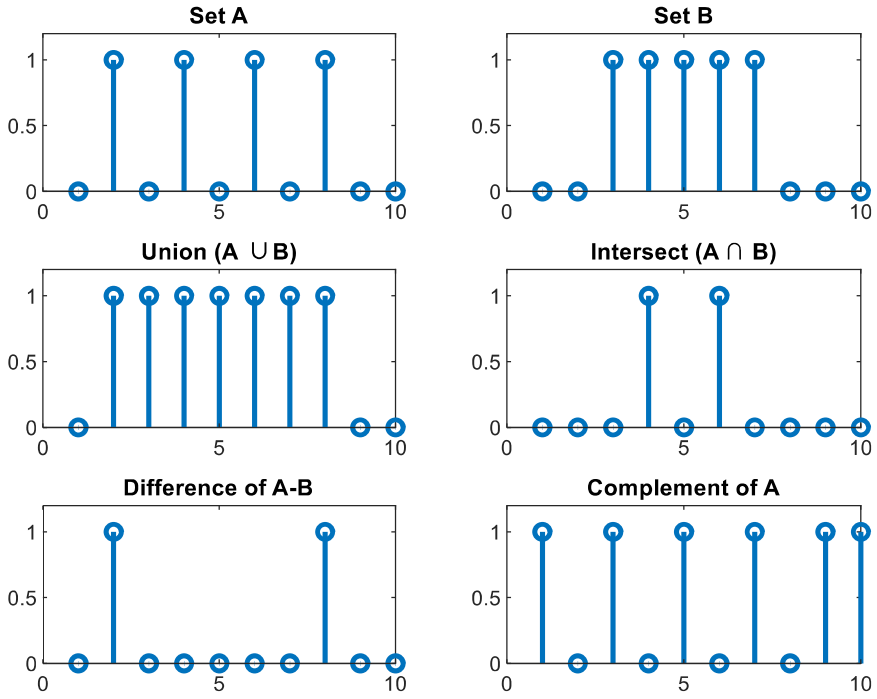
Program:

```
clc;
clear;
% Step 1: Define universe
U = 1:10;
% Step 2: Define two crisp sets
A = [2 4 6 8];
B = [3 4 5 6 7];
% Step 3: Operations
unionAB = union(A,B);
intersectAB = intersect(A,B);
diffAB = setdiff(A,B);      % A - B
compA = setdiff(U,A);       % Complement of A
% Step 4: Display results
disp(['Universe U = ', num2str(U)]);
disp(['Set A = ', num2str(A)]);
disp(['Set B = ', num2str(B)]);
disp(['Union (A ∪ B) = ', num2str(unionAB)]);
disp(['Intersection (A ∩ B) = ', num2str(intersectAB)]);
disp(['Difference (A - B) = ', num2str(diffAB)]);
disp(['Complement of A = ', num2str(compA)]);
% Step 5: Plot Venn-like visualization
figure;
subplot(3,2,1); stem(U, ismember(U,A),'LineWidth',2); title('Set A'); ylim([0 1.2]);
subplot(3,2,2); stem(U, ismember(U,B),'LineWidth',2); title('Set B'); ylim([0 1.2]);
subplot(3,2,3); stem(U, ismember(U,unionAB),'LineWidth',2); title('Union (A ∪ B)');
ylim([0 1.2]);
subplot(3,2,4); stem(U, ismember(U,intersectAB),'LineWidth',2); title('Intersect (A ∩ B)');
ylim([0 1.2]);
subplot(3,2,5); stem(U, ismember(U,diffAB),'LineWidth',2); title('Difference of A-B');
```

```
ylim([0 1.2]);
subplot(3,2,6); stem(U, ismember(U,compA),'LineWidth',2); title('Complement of A');
ylim([0 1.2]);
```

Output:

```
Universe U = 1   2   3   4   5   6   7   8   9  10
Set A = 2  4  6  8
Set B = 3  4  5  6  7
Union (A ∪ B) = 2  3  4  5  6  7  8
Intersection (A ∩ B) = 4  6
Difference (A - B) = 2  8
Complement of A = 1   3   5   7   9  10
```



Results:

The operations of crisp sets were successfully implemented in MATLAB. The union combined elements from both sets, the intersection gave common elements, the difference gave elements present in one set but not the other, and the complement gave elements from the universe not present in the set. Thus, crisp set operations were verified successfully.

Ex.No: 09 **IMPLEMENTATION OF LOGIC GATES** **Date:**

Aim:

To implement basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) using MATLAB and verify their truth tables.

Theory:

Logic gates are the fundamental building blocks of digital systems. Each gate performs a logical operation on one or more binary inputs to produce a single binary output.

Basic Logic Gate Operations:

NOT Gate: (Inverts input)

$$Y = \bar{A}$$

AND Gate: (Only 1 if both inputs are 1)

$$Y = A \cdot B$$

OR Gate: (1 if either input is 1)

$$Y = A + B$$

NAND Gate: (Only 0 if both inputs are 1)

$$Y = \overline{A \cdot B}$$

NOR Gate: (0 if either input is 1)

$$Y = \overline{A + B}$$

XOR Gate (Exclusive OR): (Outputs **1** only when the two inputs are **different)**

$$Y = A \oplus B = (A \cdot \bar{B}) + (\bar{A} \cdot B)$$

XNOR Gate (Exclusive NOR): (Outputs **1** only when the two inputs are **same)**

$$Y = \overline{A \oplus B}$$

Algorithm:

1. Start.
2. Define all possible binary inputs $A, B \in \{0,1\}$.
3. Apply logical operators $(\&, |, \sim, xor)$ in MATLAB.
4. Generate truth tables for each gate.
5. Display results.

6. Stop.

Program:

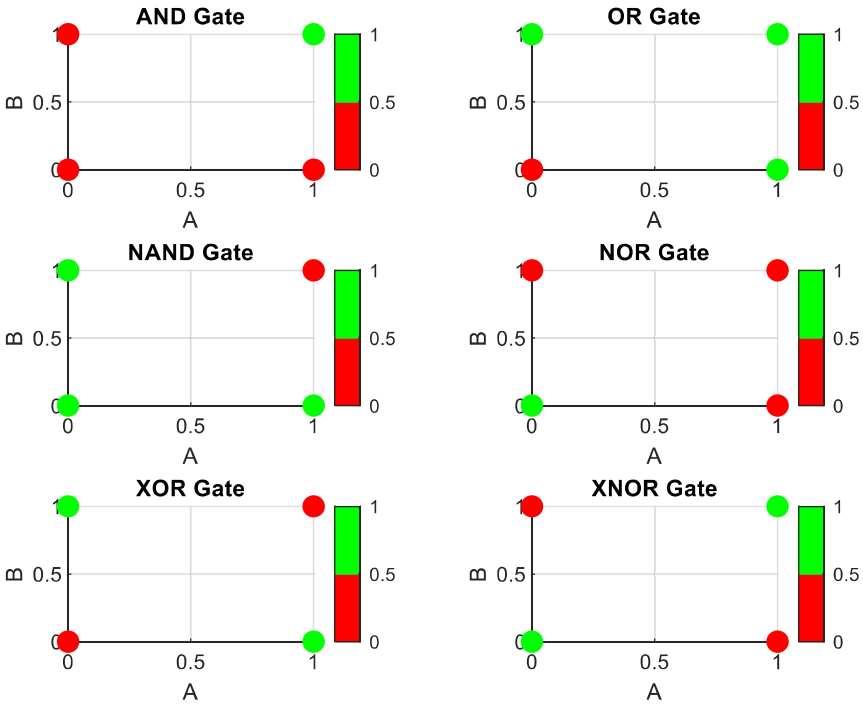
```
clc; clear;
% Step 1: Define all possible inputs
A = [0 0 1 1]';
B = [0 1 0 1]';
% Step 2: Implement logic gates
NOTA = ~A;
NOTB = ~B;
AND  = A & B;
OR   = A | B;
NAND = ~(A & B);
NOR  = ~(A | B);
XOR = xor(A, B);
XNOR = ~xor(A, B);
% Step 3: Display truth tables
T = table(A, B, NOTA,NOTB,AND,OR,NAND, NOR, XOR, XNOR);
disp('Truth Table for Logic Gates:');
disp(T);
% Step 4: Plot input combinations (A,B) and outputs
figure;
subplot(3,2,1);
scatter(A,B,80,AND,'filled');
title('AND Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,2);
scatter(A,B,80,OR,'filled');
title('OR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
```

```
colorbar;
subplot(3,2,3);
scatter(A,B,80,NAND,'filled');
title('NAND Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,4);
scatter(A,B,80,NOR,'filled');
title('NOR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,5);
scatter(A,B,80,XOR,'filled');
title('XOR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,6);
scatter(A,B,80,XNOR,'filled');
title('XNOR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
```

Output:

```
Truth Table for Logic Gates:
    A     B    NOTA     NOTB      AND      OR      NAND      NOR      XOR      XNOR
    _     _    _____    _____     _____    _____   _____     _____    _____    _____

    0     0    true     true      false    false   true      true     false    true
    0     1    true     false     false    true    true      false    true     false
    1     0    false    true      false    true    true      false    true     false
    1     1    false    false     true     true    false     false    false    true
```



Result:

The basic logic gates AND, OR, NOT, NAND, NOR, XOR, and XNOR were successfully implemented in MATLAB. Their truth tables were generated and matched with the expected outputs. The behavior of the logic gates was verified and visualized using plots.

Ex.No: 10 IMPLEMENTATION OF GENETIC ALGORITHMS Date:

Aim:

To study and implement the **Genetic Algorithm (GA)** for solving optimization problems and obtain the optimal solution with MATLAB.

Algorithm:

Initialize Population:

- Generate a random initial population of N individuals (chromosomes).
- Each chromosome represents a candidate solution.

Chromosome Encoding:

- Represent variables as binary strings or real values.
- Example: For binary GA,

$$x = x_{min} + \frac{decimal(chromosome)}{2^L - 1}(x_{max} - x_{min})$$

$where\ L\ =\ chromosome\ length.$

Fitness Evaluation:

- Evaluate the **fitness** of each chromosome using the objective function $f(x)$.
- For minimization:

$$Fitness = \frac{1}{1 + f(x)}$$

Selection:

- Select parents according to fitness values.
- Methods: **Roulette Wheel**, **Tournament**, **Rank Selection**.

Crossover (Recombination):

- With probability Pc, choose a crossover point and exchange chromosome parts between parents.
- Example (single-point crossover):

  Parent1: 10101

  Parent2: 11011

  Crossover → Child1: 10111, Child2: 11001

Mutation:

- With probability Pm, flip bits or add perturbation.

- Ensures diversity and avoids premature convergence.

New Generation Formation:

- Combine parents and offspring to form the next generation.

Termination:

- Repeat until **maximum generations** reached or solution **converges**.

Mathematical Formulas in GA

Chromosome Decoding

$$x = x_{min} + \frac{decimal(binary)}{2^L - 1}(x_{max} - x_{min})$$

Fitness Function

$$Fitness = \frac{1}{1 + f(x)} \quad \text{(for minimization)}$$

Selection Probability

$$P_i = \frac{Fitness_i}{\sum_{j=1}^{N} Fitness_j}$$

Crossover

$$Child = [Parent1(1:k), Parent2(k+1:L)]$$

Mutation

$$bit' = \neg(bit)$$

Procedure

1. Define the optimization problem (objective function, constraints, bounds).
2. Initialize GA parameters:
   - Population size (N)
   - Chromosome length (L)
   - Crossover probability (Pc)
   - Mutation probability (Pm)
   - Maximum generations
3. Generate the initial population randomly.
4. Decode chromosomes into problem variables.
5. Evaluate the fitness of each chromosome.

6. Apply selection, crossover, and mutation operators.
7. Form the next generation and repeat evaluation.
8. Continue until termination criteria are met.
9. Display the optimal solution and fitness value.

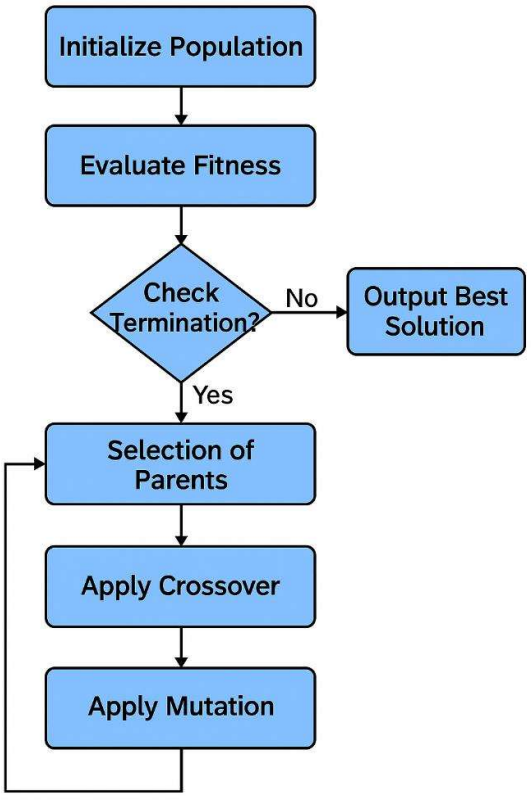
Matlab Command for GA

[x,fval] = ga(fitnessFunc, nvars, [], [], [], [], lb, ub, [], options);

GA Options

Option	Meaning
'PopulationSize'	Number of individuals in each generation
'MaxGenerations'	Maximum number of generations
'CrossoverFraction'	Proportion of crossover vs mutation
'MutationFcn'	Mutation function
'Display'	'off', 'iter', 'final'
'PlotFcn'	Plot functions like @gaplotbestf (best fitness)
'EliteCount'	Number of elite individuals preserved
'FunctionTolerance'	Stop if improvement < tolerance

Flow Chart



Program:

```
clc; clear; close all;
% Objective function
fitnessFunc = @(v) v(1).^2 + v(2).^2;   % f(x,y) = x^2 + y^2
% Number of variables
nvars = 2;
% Lower and upper bounds
lb = [-10 -10];
ub = [10 10];
% GA Options
options = optimoptions('ga', ...
   'Display','iter', ...                % show iteration details
   'MaxGenerations',100, ...            % maximum number of generations
   'PopulationSize',50, ...             % number of individuals
   'CrossoverFraction',0.8, ...         % fraction of crossover
   'MutationFcn',{@mutationgaussian,0.2}, ... % Gaussian mutation
   'OutputFcn',@myGAcontrol, ...        % Custom output function
   'PlotFcn',{@gaplotbestf,@gaplotbestindiv}); % Plots
% Run GA
[x,fval] = ga(fitnessFunc, nvars, [], [], [], [], lb, ub, [], options);
% Display results
disp(['Optimal solution: x = ', num2str(x(1)), ', y = ', num2str(x(2))]);
disp(['Minimum value: f(x,y) = ', num2str(fval)]);
% Plot contour and optimal point
[X,Y] = meshgrid(-10:0.1:10, -10:0.1:10);
Z = X.^2 + Y.^2;
figure;
contour(X,Y,Z,50); hold on;
%plot(x(1),x(2),'rp','MarkerSize',12,'MarkerFaceColor','r');
plot(x,fval,'rp','MarkerSize',12,'MarkerFaceColor','r');
title('GA Optimization Result for f(x,y) = x^2 + y^2');
xlabel('x'); ylabel('y'); grid on;
```

```
%% ======================
% Custom Output Function
% ======================
function [state,options,optchanged] = myGAcontrol(options,state,flag)
   optchanged = false;
   switch flag
      case 'iter'
         % Show population of this generation
         fprintf('Generation %d Population:\n', state.Generation);
         disp(state.Population);
         % Early stopping if best fitness below threshold
         bestFitness = min(state.Score);
         if bestFitness <= 1e-3
            fprintf('Stopping early: best fitness = %.6f\n', bestFitness);
            state.StopFlag = 'y';
         end
   end
end
```

Output:

Single objective optimization:2 Variable(s)

Options:

CreationFcn: @gacreationuniform

CrossoverFcn: @crossoverscattered

SelectionFcn: @selectionstochunif

MutationFcn: @mutationgaussian

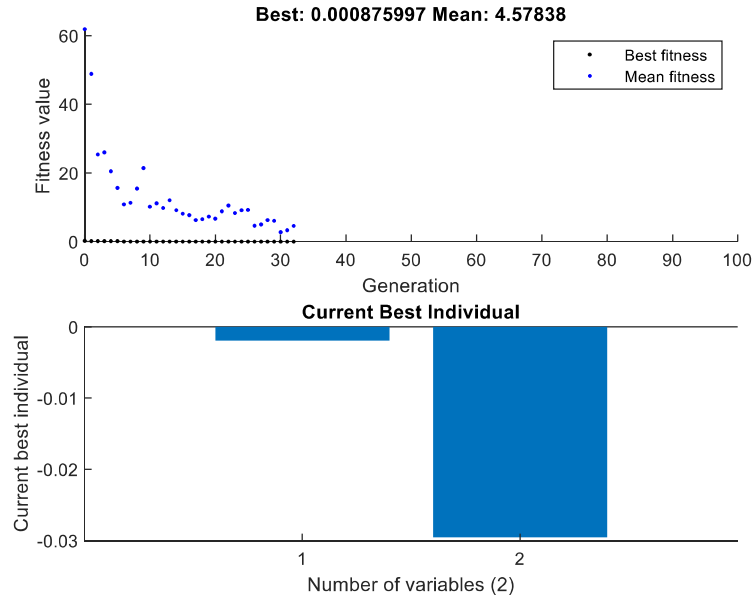
Stopping early: best fitness = 0.000876

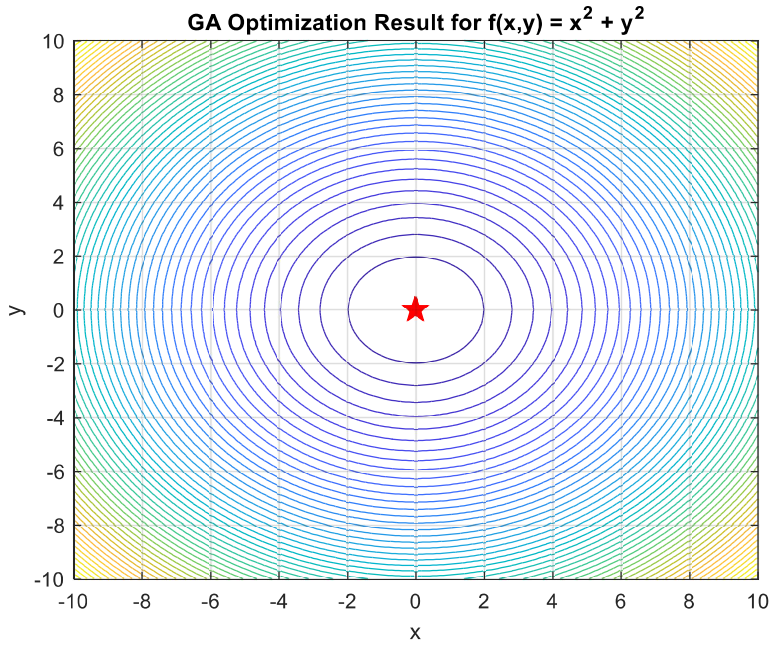
		Best	Mean	Stall
Generation	Func-count	f(x)	f(x)	Generations
32	**1557**	**0.000876**	**4.578**	**0**

Optimization terminated: y

Optimal solution: x = -0.0019322, y = -0.029534

Minimum value: f(x,y) = 0.000876





Results:

The Genetic Algorithm was applied to minimize the function $f(x, y) = x^2 + y^2$ within the range $[-10, 10]$ for both variables. The algorithm started with a random initial population and evolved solutions using selection, crossover, and Gaussian mutation. During the iterations, the best fitness value improved steadily and the algorithm successfully converged to the global minimum. The optimal solution obtained was approximately **x = -0.0019322 and y = -0.029534**, with the corresponding minimum function value **f(x,y) = 0.000876**. The contour plot confirmed that the GA correctly located the minimum point at the origin.

Ex.No: 11 IMPLEMENTATION OF CLASSIFICATION ALGORITHM Date:

Aim:

To implement and compare different classification algorithms — **Support Vector Machine (SVM), k-Nearest Neighbors (kNN), Decision Tree, Random Forest, and Neural Network** — using the Fisher's Iris dataset in MATLAB, and to evaluate their performance using confusion matrices and accuracy.

Algorithm:

1. **Load Dataset**
   - Import the Fisher's Iris dataset (load fisheriris).
   - Extract features (meas) and labels (species).
2. **Data Preprocessing**
   - Split the dataset into training (70%) and testing (30%) using cvpartition.
3. **Model Training**

   Train classifiers:
   - SVM using fitcecoc.
   - kNN using fitcknn.
   - Decision Tree using fitctree.
   - Random Forest using fitcensemble.
   - Neural Network using fitnet (with one-hot encoding).
4. **Prediction**
   - Use each model to predict the class labels of the test dataset.
5. **Evaluation**
   - Compute the accuracy = (Correct Predictions ÷ Total Test Samples) × 100.
   - Generate and analyze confusion matrices using confusionchart.

Procedure:

1. Start MATLAB and create a new script.
2. Load the Fisher's Iris dataset (load fisheriris).
3. Partition the dataset into training and testing sets.
4. Train five classifiers:
   - **SVM (fitcecoc) for multi-class classification.**
   - **kNN (fitcknn) with 5 neighbors.**
   - **Decision Tree (fitctree) for rule-based classification.**

- **Random Forest (fitcensemble) using bagged trees.**
- **Neural Network (fitnet) with 10 hidden neurons.**

5. Predict the labels for test data using each trained model.
6. Plot confusion matrices to visualize classification performance.
7. Compute accuracy values for each classifier.
8. Compare the results to identify the best classifier.

Classification & Regression

- **fitcsvm** → Fit a **binary SVM classifier**.
- **fitcecoc** → Fit a **multiclass classifier** using Error-Correcting Output Codes (ECOC) with SVM or other learners.
- **fitcknn** → Fit a **k-nearest neighbor classifier**.
- **fitctree** → Fit a **classification decision tree**.
- **fitcensemble** → Fit an **ensemble classifier** (e.g., bagged trees, boosting, random forest).
- **fitclinear** → Fit a linear model for classification.
- **fitcnb** → Fit a **Naive Bayes classifier**.
- **fitcrm** → Fit a classification model for **cost-sensitive learning** (less common).
- **fitrsvm** → Fit a **Support Vector Machine regression model**.
- **fitrlinear** → Fit a **linear regression model**.
- **fitrtree** → Fit a **regression decision tree**.
- **fitrensemble** → Fit an **ensemble regression model**.
- **fitrgp** → Fit a **Gaussian process regression model**.
- **fitrkernel** → Fit a **kernel regression model**.
- **fitrm** → Fit a **repeated measures model**.

Neural Networks (Deep Learning Toolbox)

- **fitnet** → Create and train a **neural network for function fitting** (regression).

Curve Fitting & Statistics

- **fit** → Fit **curves or surfaces** to data (from Curve Fitting Toolbox).
- **fitdist** → Fit a **probability distribution** object to data.
- **fitlm** → Fit a **linear regression model**.
- **fitglm** → Fit a **generalized linear model**.
- **fitnlm** → Fit a **nonlinear regression model**.
- **fitglme** → Fit a **generalized linear mixed-effects model**.

- **fitlme** → Fit a **linear mixed-effects model**.

Program:

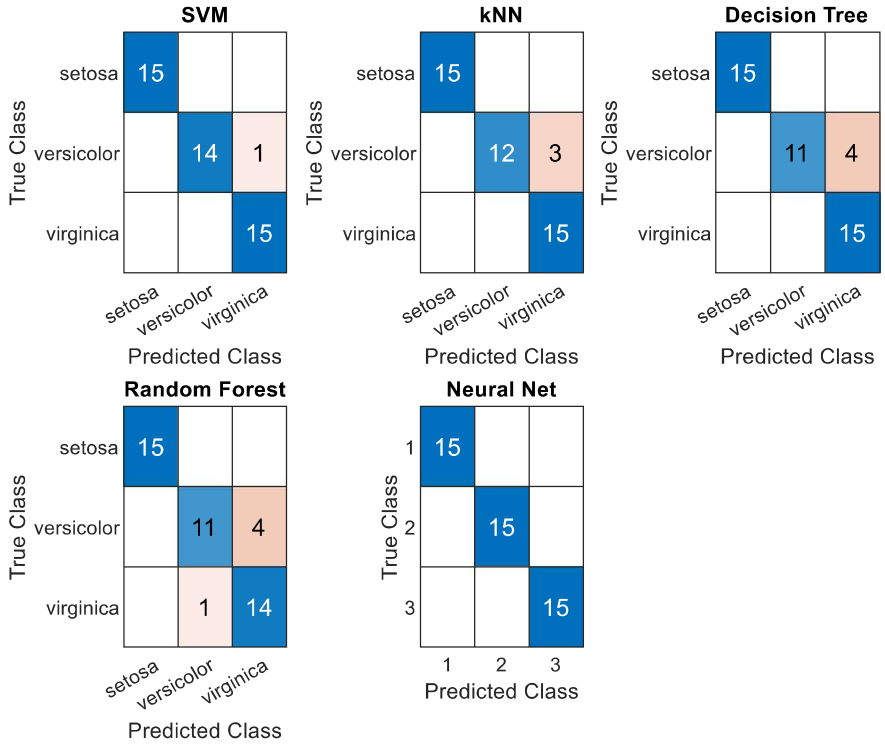
```
clc; clear; close all;
%% Load Dataset
load fisheriris
X = meas;        % Features (150x4)
Y = species;     % Labels (150x1 cell)
%% Split dataset (70% train, 30% test)
cv = cvpartition(Y,'HoldOut',0.3);
Xtrain = X(training(cv),:);
Ytrain = Y(training(cv));
Xtest  = X(test(cv),:);
Ytest  = Y(test(cv));
%% 1. Support Vector Machine (SVM)
svmModel = fitcecoc(Xtrain,Ytrain);
Ypred_svm = predict(svmModel,Xtest);
%% 2. k-Nearest Neighbors (kNN)
knnModel = fitcknn(Xtrain,Ytrain,'NumNeighbors',5);
Ypred_knn = predict(knnModel,Xtest);
%% 3. Decision Tree
treeModel = fitctree(Xtrain,Ytrain);
Ypred_tree = predict(treeModel,Xtest);
%% 4. Random Forest (Ensemble of Decision Trees)
rfModel = fitcensemble(Xtrain,Ytrain,'Method','Bag');
Ypred_rf = predict(rfModel,Xtest);
%% 5. Neural Network using fitnet
X_nn = X';              % transpose for neural net (features x samples)
Ynum = grp2idx(Y);      % convert species to numeric (1,2,3)
Y_onehot = full(ind2vec(Ynum')); % one-hot encoding
% Split for train/val/test
net = fitnet(10);
net.divideParam.trainRatio = 0.7;
```

```
net.divideParam.valRatio   = 0.15;
net.divideParam.testRatio  = 0.15;
[net,~] = train(net,X_nn,Y_onehot);   % train NN
Ypred_nn = net(Xtest');               % test NN
[~,Ypred_nn_class] = max(Ypred_nn);   % get class labels
Ypred_nn = categorical(Ypred_nn_class)';
Ytest_nn = categorical(grp2idx(Ytest));
%% Confusion Matrices
figure;
subplot(2,3,1); confusionchart(Ytest,Ypred_svm); title('SVM');
subplot(2,3,2); confusionchart(Ytest,Ypred_knn); title('kNN');
subplot(2,3,3); confusionchart(Ytest,Ypred_tree); title('Decision Tree');
subplot(2,3,4); confusionchart(Ytest,Ypred_rf); title('Random Forest');
subplot(2,3,5); confusionchart(Ytest_nn,Ypred_nn); title('Neural Net');
%% Accuracy Results
acc_svm  = sum(strcmp(Ypred_svm,Ytest))/length(Ytest)*100;
acc_knn  = sum(strcmp(Ypred_knn,Ytest))/length(Ytest)*100;
acc_tree = sum(strcmp(Ypred_tree,Ytest))/length(Ytest)*100;
acc_rf   = sum(strcmp(Ypred_rf,Ytest))/length(Ytest)*100;
acc_nn   = sum(Ypred_nn == Ytest_nn)/length(Ytest)*100;
fprintf('Accuracy (SVM): %.2f%%\n',acc_svm);
fprintf('Accuracy (kNN): %.2f%%\n',acc_knn);
fprintf('Accuracy (Decision Tree): %.2f%%\n',acc_tree);
fprintf('Accuracy (Random Forest): %.2f%%\n',acc_rf);
fprintf('Accuracy (Neural Network): %.2f%%\n',acc_nn);
```

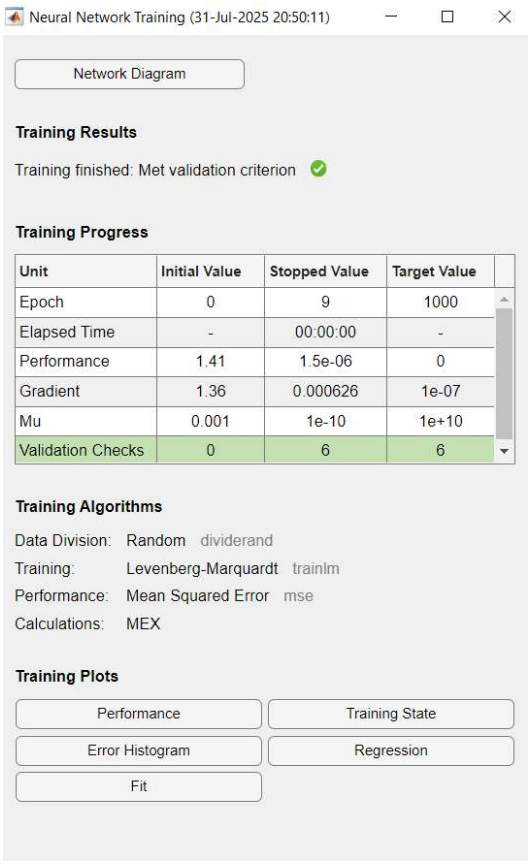
Output:

```
Accuracy (SVM): 97.78%
Accuracy (kNN): 93.33%
Accuracy (Decision Tree): 91.11%
Accuracy (Random Forest): 88.89%
Accuracy (Neural Network): 100.00%
```

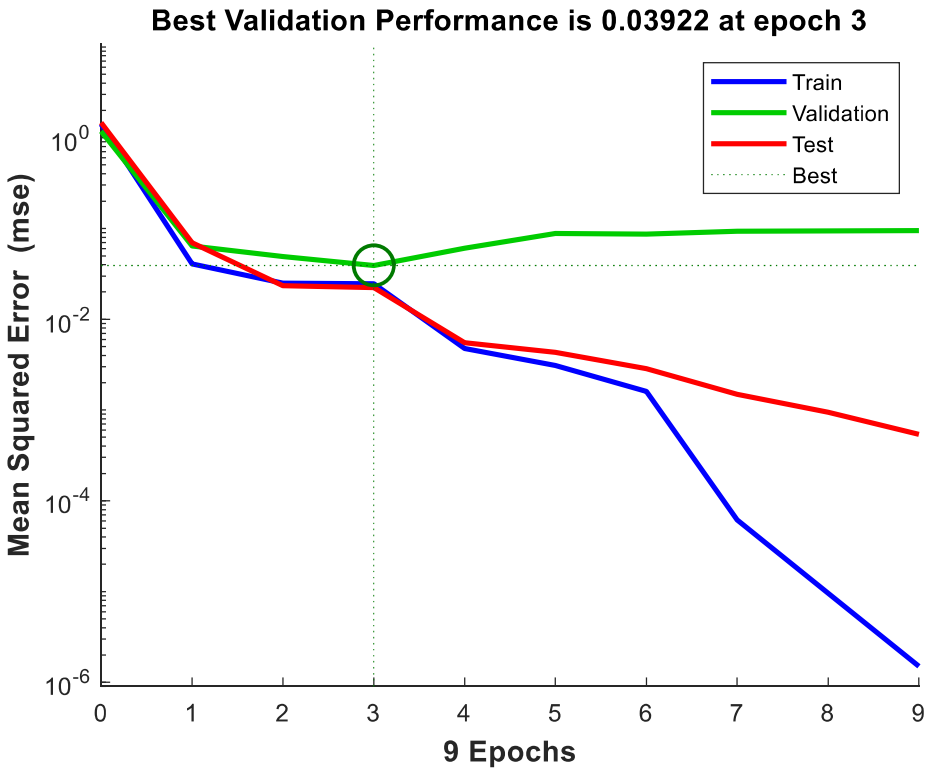
Confusion Matrices:



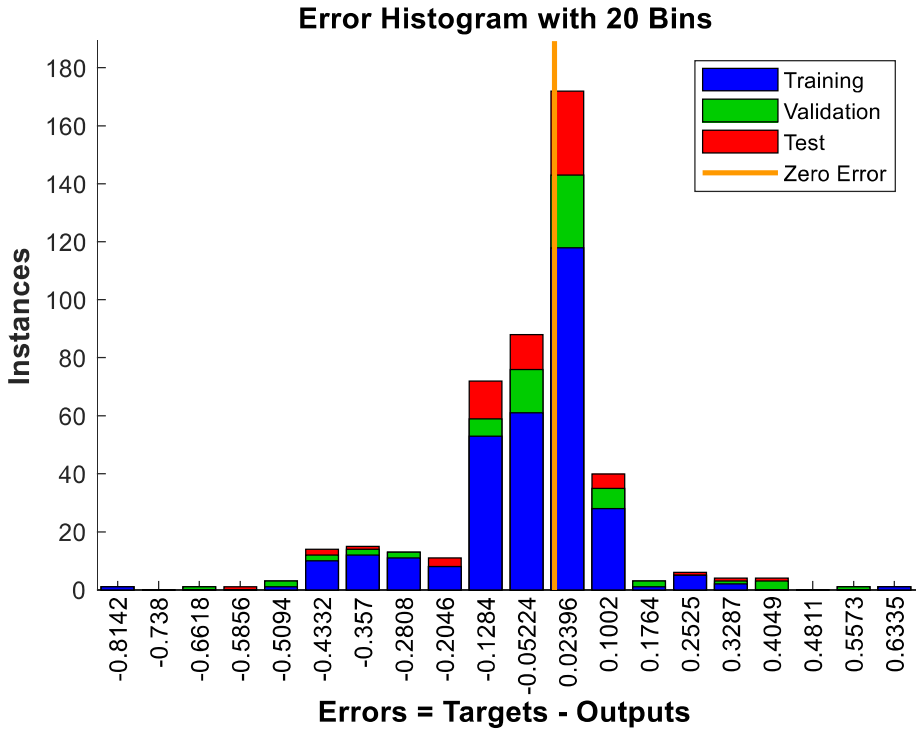
Network Diagram:



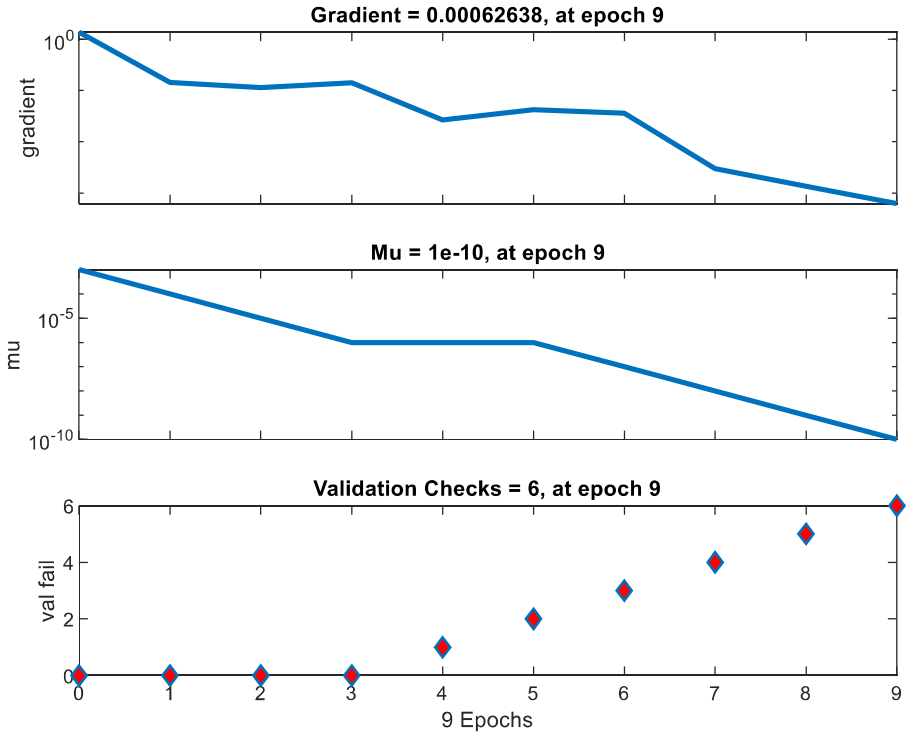
Training Performance:



Neural Network Training Error:



Training State:



Results:

The classification algorithms were implemented on the Fisher's Iris dataset in MATLAB and their performances were evaluated. The **Support Vector Machine (SVM)** achieved an accuracy of **97.78%**, while the **k-Nearest Neighbors (kNN)** classifier obtained an accuracy of **93.33%**. The **Decision Tree** classifier gave a lower accuracy of **91.11%**, and the **Random Forest** model achieved **88.89%**. The **Neural Network (fitnet)** outperformed all other models with a perfect accuracy of **100.00%**. From these results, it can be concluded that the Neural Network provided the best classification performance, followed by SVM, while the Random Forest had the lowest accuracy among the tested classifiers.